CS2323 Computer Architecture · IIT Hyderabad

Vector Extension Support for the Kites RISC-V Simulator

Validation Report

RISC-V "V" extension 1.0, the Zve* subsets, and the Zvfh, Zvbb/Zvkb and Zvbc extensions

Document	Validation report
Version	1.0
Author	Raam M
Repository	`https://github.com/Raam-M/Kites`, branch `vector-extension`
Environment	macOS 26 (arm64), Apple Clang 15, Qt 6, Spike 1.1.1-dev, LLVM `llvm-mc`
Companions	Scope document (requirements, quality gates); design document

Purpose of this document

This report records how the vector extension was validated against the plan in the scope document, and the results: unit and integration tests, differential testing against Spike, coverage, performance, the GUI checks, the defects found, a requirement-to-test traceability matrix and the status of each quality gate.

Contents

1 Summary **2**

2 Validation levels **2**

3 Automated tests **3**

4 Differential testing against Spike **4**

4.1 Harness 4

4.2 Results 4

4.3 Allowed differences 5

5 Coverage **5**

6 Performance of scalar code **6**

7 GUI checks **7**

8 Defects **10**

8.1 In the existing Kites code 10

8.2 In the new code 10

9 Traceability **10**

10 Quality gates **12**

11 Limitations **12**

12 Reproducing the results **12**

1 Summary

Table 1: Results at a glance.

Measure	Result
Automated tests (CTest)	261 of 261 pass: 103 new vector and pipeline tests, 158 existing tests
Instructions covered	647 table entries (627 V, 20 Zvbb/Zvkb/Zvbc), each with an `llvm-mc` golden encoding; semantic tests for every instruction family
Differential testing vs Spike	2 405 programs, 0 mismatches: 55 example runs and 2 350 random programs over 20 configurations (Table 4)
Line / branch / function coverage	95.2 % / 88.5 % / 98.7 % of the new vector code (targets: 90 % lines, 75 % branches)
Scalar performance (NF-01)	per instruction faster than before (CPU time -4.4 % and -6.4 % on the longer programs); about 3 ms more start-up (Section 6)
Quality gates	QG1–QG4, QG6 and QG7 met; QG5 met on macOS, Linux not run (Section 10)

All compulsory requirements (tiers A1 and A2) and all stretch goals (B1–B10) are implemented and verified. Validation found nine defects in the existing Kites code, six of which are fixed (Section 8). The differences from Spike that the specification allows are handled explicitly (Section 4.3).

2 Validation levels

Table 2: Validation levels of the scope document and what was done.

Level	Focus	Done	Status
L1	Units	vtype, VLMAX, legality helpers, rounding, saturation, `rec7`/`rsqrt7`, high multiplies, half rounding, configuration parsing, element formatting	**pass**
L2	Assembler	1 293 `llvm-mc` golden encodings; decode and disassembly round trip; error messages; labels named like vtype fields; pseudo-instructions	**pass**
L3	Semantics	every family against an independent reference model over all 22 SEW/LMUL pairs, `vl` values, masks, policies, `vstart`; FP edge values; half precision against hardware `_Float16`	**pass**
L4	Legality	every rule raises an error and leaves all state bit-identical	**pass**
L5	Programs	11 example programs checked against expected values and scalar versions, at every VLEN and on the pipelines	**pass**
L6	Differential	Spike, 2 405 programs over 20 configurations (Section 4)	**pass**
L7	System	undo/redo round trips (single-cycle and pipeline), reset, retirement counting, statistics undo	**pass**
L8	Regression	full existing suite; macOS	**partial**
L9	Coverage	clang source-based coverage (Section 5)	**pass**
L10	GUI	offscreen renders of the new views, checklist (Section 7)	**pass**

3 Automated tests

Table 3 lists the new test suites. Test names begin with the requirement they verify (for example `VM02_…` for VM-02). Expected values come from an independent reference model (`test/vector/vector_reference.h`), from `llvm-mc`, from the hardware's `_Float16` conversions, from hand calculation, or from the single-cycle processor when a pipeline is checked.

Table 3: New test suites (`test/vector/`, `test/rv5s_regression_test.cpp`).

Suite	Tests	Covers	Oracle
VecAsm	12	golden encodings, decode, round trip, inventory, syntax errors, vtype forms, labels, CSR and vector pseudos	`llvm-mc`
VecConfig	7	reset state, `vset*`, `x0` cases, `vill`, vector CSRs, `vstart`	spec rules
VecInteger	9	single-width, division, multiply-add, widening, narrowing, extension, carry, compare, merge	reference model
VecPermute	7	reductions, mask operations, `viota/vid`, scalar moves, slides, gather, compress, whole-register moves	reference model
VecMemory	5	unit-stride, strided, indexed, mask, whole-register, fault-only-first, cache traffic	reference model
VecFloat	8	FP arithmetic, FMA, compares, special values and flags, conversions, estimates, reductions, legality	host IEEE, spec tables
VecHalf	5	Zvfh in all rounding modes, FMA forms, integer conversions, flags; element formatting	hardware `_Float16`
VecExtensions	11	Zvbb, Zvbc, disabled extensions, Zve* profiles, option parsing, Zvfh values, VLEN, agnostic fill	reference model
VecLegality	4	group, overlap, `vill`, `vstart` rules; state unchanged	spec rules
VecSystem	2	undo/redo round trip, retirement	recorded state
VecExamples	11	the example programs	expected values, scalar code
VecPipeline	10	examples and dependent chains on the pipelines, stall cycles, NOP rule, memory order, flushing, errors in WB, undo/redo, FP ordering	single-cycle model
VecProfiler	5	statistics, classification, hand-computed schedules, structural hazards, undo	hand calculation
VectorCustomPseudo	3	vector expansions: accepted, rejected, expanded and run	expected values
Rv5sRegression	4	AUIPC/LUI, `la`, FP integer-source forwarding, draining	expected values

Mutation checks. To confirm that the tests detect defects, deliberate faults were inserted and the tests were run, then removed. For example, removing the stall for scalar instructions that depend on a vector result made four pipeline tests fail; in one of them a strip-mined loop no longer terminated, which the test reports through its step limit.

4 Differential testing against Spike

4.1 Harness

`tools/vector/spike_diff.py` runs the same program on Kites and on Spike [2]. A shared epilogue stores the final architectural state into a dump area next to the program data: `x1`–`x31`, `f0`–`f31`, `v0`–`v31`, `vl`, `vtype`, `vxsat`, `vxrm`, `fflags` and `frm`. Kites runs the program through `kites-cli` and dumps memory; Spike runs an ELF built with `llvm-mc` and `ld.lld` and writes the same region with `+signature`. The data memory, vector registers, vector CSRs, `fflags` and the scalar registers the program uses are compared byte for byte. The harness selects the Kites processor model, VLEN, profile and extensions, and passes the matching ISA string to Spike (for example `rv64gc_zve64f_zfhmin_zvfh_zvl128b`).

Random programs are generated from a seed. A prologue loads random data with edge values (zeros, all ones, sign boundaries, infinities, NaNs, subnormals) into all vector and FP registers and sets the scalar operands; the body draws from every instruction family: integer, multiply-add, compares, widening, narrowing, extension, carry, merges, reductions, mask operations, permutations, fixed-point rounding modes, floating point (including conversions at every legal width and rounding modes), loads and stores of every kind, `vset*` with random AVL, SEW and LMUL, and the Zvbb/Zvkb/Zvbc instructions. Only instructions legal for the target configuration are generated. A failing program can be bisected to the first line where the states diverge.

4.2 Results

Table 4: Final campaign: every program matches Spike.

Run	Processor	VLEN	Profile, extensions	Programs	Matching
examples	single-cycle	128	V	11	11
examples	single-cycle	512	V	11	11
examples	H_NF	128	V	11	11
examples	H_F	128	V	11	11
examples	H_F	1024	V	11	11
random	single-cycle	128	V	500	500
random	single-cycle	256	V	150	150
random	single-cycle	512	V	150	150
random	single-cycle	1024	V	150	150
random	single-cycle	128	V, Zvfh, Zvbb, Zvbc	300	300
random	single-cycle	256	V, Zvfh, Zvbb, Zvbc	100	100
random	single-cycle	128	V, Zvkb	50	50
random	single-cycle	128	Zve64d	100	100
random	single-cycle	128	Zve64f, Zvfh	100	100
random	single-cycle	128	Zve64x, Zvbb, Zvbc	100	100
random	single-cycle	128	Zve32f, Zvfh	100	100

Table 4: Final campaign (continued).

Run	Processor	VLEN	Profile, extensions	Programs	Matching
random	single-cycle	128	Zve32x, Zvkb	100	100
random	H_NF	128	V	150	150
random	H_F	128	V	200	200
random	H_F	256	V, Zvfh, Zvbb, Zvbc	100	100
total				2 405	2 405

Each random program has 40 instructions after the prologue (about 60 to 120 executed instructions including the prologue, the epilogue and expansions). The models without hazard detection (NH_NF, NH_F) are not in the random campaign, because the random programs do not contain the NOPs those models need; they are covered by `PL02` and the existing pipeline tests.

4.3 Allowed differences

The specification lets implementations differ in some places. The harness avoids these cases, or Kites makes the same choice as Spike:

Agnostic elements
Kites leaves them undisturbed by default, as Spike does; the all-ones fill is verified by unit tests (VC-04).

Misaligned element accesses
Spike traps on a misaligned strided element access, Kites allows it (D8). Random strides are multiples of the element size.

NaN-boxing
Spike treats a single or half value that is not NaN-boxed as the canonical NaN; Kites reads the low SEW bits (F18). The harness loads NaN-boxed scalars, which both read the same way. FP registers are not compared.

Unordered sums
Both add in element order. With no active element and a NaN scalar, Spike returns the canonical NaN; Kites originally returned the scalar unchanged. Both are allowed; Kites now makes the same choice (found by the campaign at VLEN=256, seed 6062).

5 Coverage

The test suite was built with clang source-based coverage (`-fprofile-instr-generate -fcoverage-mapping`) and the profiles were merged with `llvm-profdata`. The project's existing `coverage` target links GCC's `libgcov`, which Apple Clang does not provide, so it was not used. Coverage counts only the unit and integration tests; the Spike campaigns run through `kites-cli` and add to the confidence but not to these numbers.

Table 5: Coverage of the new vector code.

File	Lines	Branches	Functions
`vector_exec_integer.cpp`	99.7 %	98.3 %	100 %
`vector_unit.cpp`	97.0 %	92.1 %	100 %
`vector_exec_memory.cpp`	94.7 %	90.6 %	100 %
`vector_exec_float.cpp`	92.0 %	81.0 %	98.4 %
`vector_isa.cpp`	92.7 %	89.2 %	92.9 %
`vector_config.cpp`	94.9 %	92.2 %	100 %
`vector_profiler.cpp`	98.5 %	91.5 %	93.3 %
`vector_format.cpp`	98.9 %	90.0 %	100 %
`vtype.cpp`	95.0 %	89.3 %	100 %
`v_formats.cpp` (assembler)	90.3 %	78.6 %	100 %
headers	100 %	100 %	100 %
total	**95.2 %**	**88.5 %**	**98.7 %**

The uncovered code is mostly error paths that need malformed input the assembler already rejects, defensive checks for states that cannot arise (such as an unknown instruction reaching a family handler), and some FP special-case branches (for example `vfrec7` of a subnormal under each rounding mode) that the Spike campaigns exercise.

6 Performance of scalar code

Requirement NF-01 asks that scalar programs run no more than 5 % slower than before the vector work. The `kites-cli` of the baseline commit `7ddb6f3` and of this branch were built in the same way (Debug, Apple Clang) and run alternately in batch mode on the single-cycle processor. Table 6 gives the median CPU time (user plus system) of each program. The pipelines could not be compared, because the baseline CLI never finishes on a pipeline model (F22).

Table 6: Scalar run time, baseline and current (median CPU time).

Program	Runs	Baseline (ms)	Current (ms)	Change
cache_matrix_mul_3x3.s	21	206	197	-4.4 %
pipeline_test_code_1.s	21	37	39	+7.3 %
scalar loop (40 002 instructions)	5	4647	4350	-6.4 %

Kites writes `registers_dump.json` after every step. The first measurement showed scalar programs up to 65 % slower, because the dump had grown by the vector registers and the vector CSRs and was formatted value by value through stream manipulators. The dump is now built in one string with a direct hexadecimal formatter (byte-identical for the existing sections) and writes vector registers without leading zeros. Per instruction, the current build is now faster than the baseline. The remaining difference on the shortest program is a one-off start-up cost of about 3 ms for building the vector instruction tables; its wall time is unchanged.

7 GUI checks

The GUI parts were rendered offscreen (Qt `offscreen` platform) from the real widgets and scenes, linked against the Kites library, after stepping `examples/vector/rgb_to_grey.s` up to its last instruction.

Table 7: GUI checklist.

Check	Result
Vector tab shows `v0`–`v31` as elements of the current SEW, in the chosen format	**pass**
LMUL groups are labelled and banded; masked-off elements are shaded; tail elements are dimmed	**pass**
Vector CSRs (decoded `vtype`) and the statistics/timing report are shown	**pass**
Settings page edits VLEN, policy, profile, extensions and timing options	**pass**
Circuit view of every processor has the vector unit; its wires light up for a vector instruction	**pass**
Register panel has Vector and Vector CSRs sections; `vtype` is decoded	**pass**
Editor highlights vector mnemonics	**pass**
An illegal vector instruction leaves the PC on it, so the error is shown at its line (PL07, VL tests)	**pass**

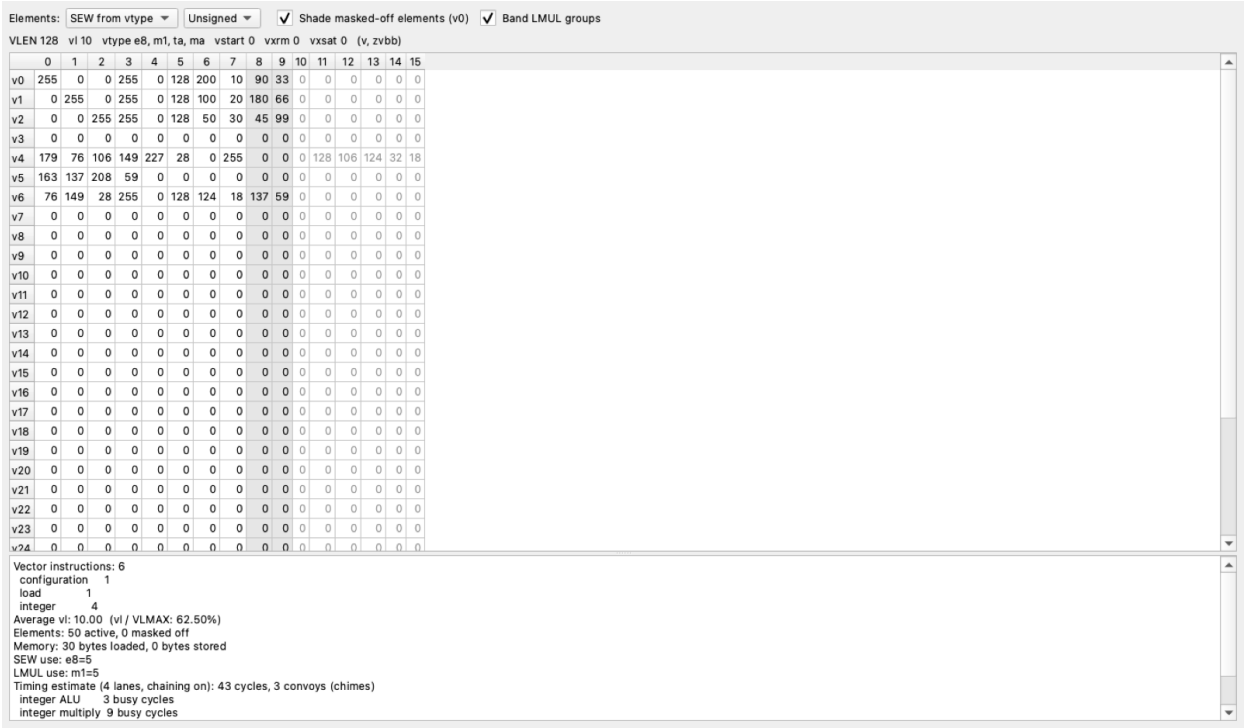
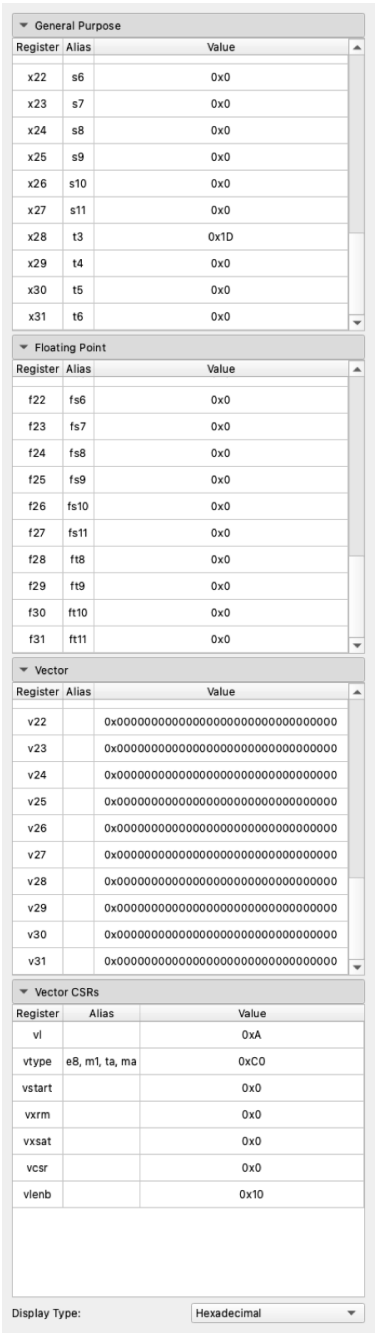


Figure 1: Vector tab in `rgb_to_grey.s` (unsigned, mask shading on). `vlseg3e8.v` has split the pixels into R, G and B in `v0`–`v2`; elements at or beyond `vl` = 10 are dimmed; elements 8 and 9 are shaded because their `v0` bits are 0; the statistics and timing estimate are below.



General Purpose

Register	Alias	Value
x22	s6	0x0
x23	s7	0x0
x24	s8	0x0
x25	s9	0x0
x26	s10	0x0
x27	s11	0x0
x28	t3	0x1D
x29	t4	0x0
x30	t5	0x0
x31	t6	0x0

Floating Point

Register	Alias	Value
f22	fs6	0x0
f23	fs7	0x0
f24	fs8	0x0
f25	fs9	0x0
f26	fs10	0x0
f27	fs11	0x0
f28	ft8	0x0
f29	ft9	0x0
f30	ft10	0x0
f31	ft11	0x0

Vector

Register	Alias	Value
v22		0x00000000000000000000000000000000
v23		0x00000000000000000000000000000000
v24		0x00000000000000000000000000000000
v25		0x00000000000000000000000000000000
v26		0x00000000000000000000000000000000
v27		0x00000000000000000000000000000000
v28		0x00000000000000000000000000000000
v29		0x00000000000000000000000000000000
v30		0x00000000000000000000000000000000
v31		0x00000000000000000000000000000000

Vector CSRs

Register	Alias	Value
vl		0xA
vtype	e8, m1, ta, ma	0xC0
vstart		0x0
vxrm		0x0
vxsat		0x0
vcsr		0x0
vlenb		0x10

Display Type: Hexadecimal

Figure 2: Register panel, scrolled to the vector sections.

```
# rgb_to_grey.s - segment loads: convert 10 packed RGB pixels to 8-bit grey.
# vlseg3e8.v splits interleaved R,G,B bytes into v0, v1, v2. The weighted sum
# 77*R + 150*G + 29*B is accumulated at 16 bits (widening multiply-add) and narrowed
# with a shift right by 8.
# Result: grey[i] = (77*R + 150*G + 29*B) >> 8
.data
rgb:  .byte 255, 0, 0,  0, 255, 0,  0, 0, 255,  255, 255, 255,  0, 0, 0
      .byte 128, 128, 128,  200, 100, 50,  10, 20, 30,  90, 180, 45,  33, 66, 99
grey: .zero 10
.text
    li   a0, 10
    la   a1, rgb
    la   a2, grey
    li   t1, 77
    li   t2, 150
    li   t3, 29
    vsetvli t0, a0, e8, m1, ta, ma
    vlseg3e8.v v0, (a1)          # v0 = R, v1 = G, v2 = B
    vwmulu.vx v4, v0, t1         # v4..v5 (16-bit) = 77*R
    vwmaccu.vx v4, t2, v1        # += 150*G
    vwmaccu.vx v4, t3, v2        # += 29*B
    vnsrl.wi v6, v4, 8           # >> 8, back to 8 bits
    vse8.v v6, (a2)

```

Figure 3: Editor with vector code.



Figure 4: Settings → Vector Extension.

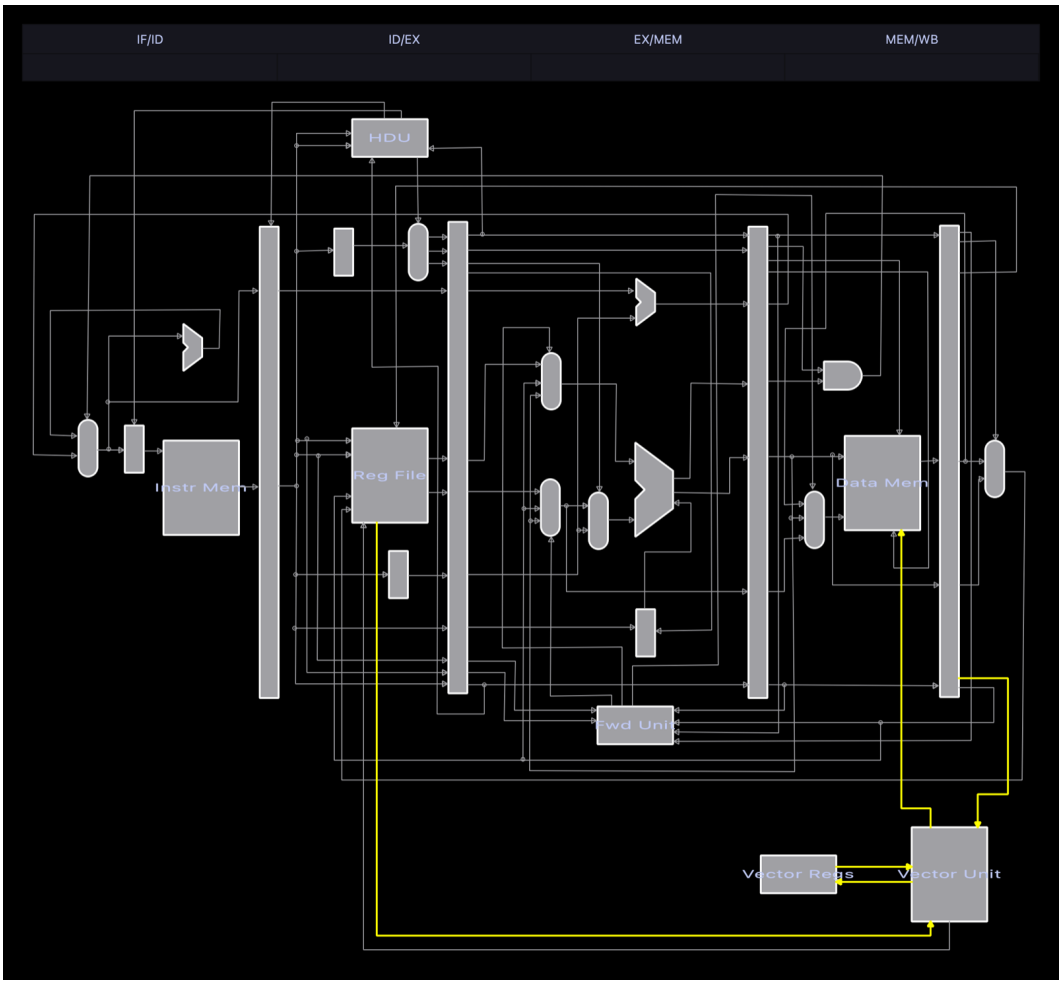


Figure 5: Circuit view of the pipeline with hazard detection and forwarding; the vector unit and its active wires (yellow) are at the bottom right.

8 Defects

8.1 In the existing Kites code

Validation found the following defects in code that existed before this work. F16–F19 are listed in the scope document; F20–F25 were found while validating the stretch goals.

Table 8: Defects in the existing code.

ID	Defect	Resolution
F16	Scalar FP: $x/0$ gives NaN, flag bits reversed, NaN-to-integer not saturated, no RMM.	Reported; vector FP does not use these routines.
F17	A label named `data`, `text` or `bss` hangs or breaks the assembler.	Reported; examples avoid the names.
F19	`test/test_main.cpp` includes `windows.h` unconditionally.	Fixed.
F20	Pipelines: AUIPC adds two register values, so `la` gives wrong addresses; LUI is not sign-extended.	Fixed, regression tests.
F21	Forwarding pipelines do not forward the integer source of F/D instructions (load/store base, `fmv.w.x`, `fcvt.s.w`). Found by the Spike campaign on H_F (17 of 150 programs).	Fixed, regression test.
F22	Pipelines pause at the end-of-program breakpoint while draining, so `kites-cli` never finishes on a pipeline model.	Fixed, regression test.
F23	CSR instructions are no-ops on the pipelines (the execute stage returns early, which also repeats the previous instruction's MEM/WB on NH_NF).	Fixed: CSRs execute in WB.
F24	Custom pseudo-instructions: an operand list written as "`a, b`" is split into an empty operand, so the line is not expanded.	Fixed, tests.
F25	The CMake `coverage` target links `libgcov`, which Apple Clang lacks.	Reported; source-based coverage used instead.

8.2 In the new code

Defects in the vector code found during validation were fixed before the corresponding commit, each with a test. The ones worth recording are: the unordered-sum NaN choice (Section 4.3); the pipeline stall needed for scalar FP instructions that follow a vector or CSR instruction, so that `fflags` and `frm` are accessed in program order (PL09); and the display of an illegal `vtype`, which the REPL and the viewer printed as a raw number and now show as `vill`.

9 Traceability

Table 9: Requirements of the scope document and the stretch goals, with their tests.

ID	Requirement (short)	Tests	Result
VS-01	32 vector registers of VLEN bits	VecConfig, VC03_*	**pass**
VS-02/03	vector CSRs, read-only rules, `vcsr`	VS02_VS03_vector_csrs	**pass**

Table 9: Traceability (continued).

ID	Requirement (short)	Tests	Result
VS-04	reset state	VS04_reset_state	pass
VC-01/02/03	`vset*`, `x0` cases, `vill`	VC01_*, VC02_*, VC03_*	pass
VM-01/02	unit-stride, strided, indexed	VM01_*, VM01_VM02_*	pass
VM-03	mask and whole-register	VM03_*	pass
VM-04	through the memory controller, undo	VM04_*, SI02_*	pass
VM-05	segment, fault-only-first	VM05_*, examples	pass
VA-01	integer, reduction, mask, permutation	VA01_* (16 tests)	pass
VA-02	division edge cases	VA02_*	pass
VA-03	fixed point	VA01_VA03_*, fir_q15	pass
VA-04	floating point	VA04_*	pass
VA-05	7-bit estimates	VA05_*	pass
VP-01	masking and policies	VA01_*, VC04_*	pass
VP-02	`vstart`	VP02_*, VL02_*	pass
VL-01/02	legality, state unchanged	VL01_*, VL02_*, PL07	pass
AS-01/02	encodings, round trip	AS01_*, AS02_*	pass
AS-03	error messages	AS03_*	pass
AS-04	labels named like vtype fields	AS04_*	pass
AS-05	highlighting, completion	GUI checklist	pass
AS-06	pseudo-instructions	AS06_*	pass
SI-01	retirement	SI01_*	pass
SI-02	undo/redo	SI02_*, PL08	pass
SI-03	scalar F/D unaffected	existing FP suites, VecAsm	pass
SI-04	pipelines execute vector code (B1)	SI04_*, PL01–PL09	pass
OB-01/02/03	dump, register view, profiler type	examples, GUI checklist, VecAsm	pass
NF-01	scalar performance	Section 6	pass
NF-02	builds; portable 128-bit multiply	macOS build; helpers	partial
NF-03	conventions, comments	review	pass
NF-04	coverage	Section 5	pass
B1	pipelined execution	VecPipeline, Spike on H_NF/H_F	pass
B2	timing model	VT01_*	pass
B3	vector viewer	GUI checklist	pass
B4	VLEN, agnostic fill	VC03_*, VC04_*, Spike at 4 VLENs	pass
B5	statistics	VS01_*	pass
B6	random differential testing	Section 4	pass
B7	circuit view	GUI checklist	pass
B8	Zvfh, Zvbb/Zvkb, Zvbc, Zve*	VX01–VX04, VecHalf, Spike	pass
B9	CLI and REPL commands	manual run, Spike harness	pass
B10	custom vector pseudo-instructions	VectorCustomPseudo	pass

10 Quality gates

Table 10: Quality gates of the scope document.

Gate	Evidence	Status
QG1	every table entry has a golden encoding (inventory test); every instruction family has semantic tests and appears in the random Spike programs	**pass**
QG2	261/261 tests pass; every deviation is documented (design document, limitations)	**pass**
QG3	95.2 % lines, 88.5 % branches	**pass**
QG4	all examples match Spike on three processors and four VLENs	**pass**
QG5	existing suite passes on macOS; Linux and Windows were not available	**partial**
QG6	undo/redo round trips for the examples on the single-cycle processor and a pipeline	**pass**
QG7	GUI checklist and renders (Section 7)	**pass**

11 Limitations

The known limitations are listed in the design document: RMM arithmetic for single and double precision rounds ties to even, underflow for those widths follows the host's tininess rule, scalar single and half values are not NaN-boxed, there are no traps, the pipeline models without hazard detection need two instructions between a vector or CSR result and its scalar use, and the timing model is an estimate. Validation used one platform (macOS on arm64).

12 Reproducing the results

```
# tests
cmake -S . -B build/test -G Ninja -DENABLE_TESTS=ON -DDISABLE_GUI=ON -DCMAKE_PREFIX_PATH=$(brew --prefix qt)
cmake --build build/test && (cd build/test && ctest)

# differential testing (needs spike, llvm-mc, ld.lld)
python3 -I tools/vector/spike_diff.py --kites build/test/kites-cli examples examples/vector/*.s
python3 -I tools/vector/spike_diff.py --kites build/test/kites-cli random --count 500 --seed 1
python3 -I tools/vector/spike_diff.py --kites build/test/kites-cli --vm rv5-h-f --vlen 256 \
    --vector-ext v,zvfh,zvbb,zvbc random --count 100 --seed 17001

# coverage (Apple Clang)
cmake -S . -B build/cov -G Ninja -DENABLE_TESTS=ON -DDISABLE_GUI=ON -DCMAKE_PREFIX_PATH=$(brew --prefix qt) \
    -DCMAKE_CXX_FLAGS="-fprofile-instr-generate -fcoverage-mapping" \
    -DCMAKE_C_FLAGS="-fprofile-instr-generate -fcoverage-mapping" -DCMAKE_EXE_LINKER_FLAGS="-fprofile-instr-
     generate"
cmake --build build/cov --target kites_test
(cd build/cov && LLVM_PROFILE_FILE="$PWD/prof/%p.profraw" ctest)
xcrun llvm-profdata merge -sparse build/cov/prof/*.profraw -o build/cov/kites.profdata
xcrun llvm-cov report build/cov/test/kites_test -instr-profile=build/cov/kites.profdata \
    src/processor/vector src/assembler/parse_formats/v_formats.cpp
```

References

[1] RISC-V International. *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture*, "V" Standard Extension for Vector Operations, version 1.0. https://docs.riscv.org/reference/isa/v20260120/unpriv/v-st-ext.html

[2] RISC-V Software. *Spike RISC-V ISA Simulator*. https://github.com/riscv-software-src/riscv-isa-sim

[3] LLVM Project. *User Guide for RISC-V Target* and `llvm-mc`. https://llvm.org/docs/RISCVUsage.html